

Sleeping Machines

AI that learns what to compute, when to compute it, and what to remember.

A new trainable architecture that already beats small Transformer baselines at a fraction of the compute.

Tero Keski-Valkama | Sole founder

Raising €3M to scale research and engineering
Research-stage deep tech · 4 October 2026



Capacity beyond activity

Beats the saved Transformer baselines at a fraction of the compute.

Wikipedia text (text8); same data and test set for all. Error: bits/char (bpc) or sMAPE %; lower is better.

Comparison	Their error	Our error	Result and compute we used
vs Transformer, 4 layers, 4 passes (10M chars)	1.908 bpc	1.888 bpc	Better · 40% of its training, 18% of its inference compute
vs Transformer, 2 layers, 1 pass (10M chars)	2.427 bpc	1.955 bpc	Better · 96% of its training, 16% of its inference compute
vs LSTM-256 (10M chars)	2.171 bpc	1.955 bpc	Better at 88% of its inference compute (more training)
vs LSTM-512, 6 passes (10M chars): queued rerun	1.799 bpc	1.888 bpc	Behind by 0.09 at 81% of its training compute
vs best tuned LSTM so far (2 tuned runs)	1.826 bpc	1.888 bpc	Behind by 0.06; it used 92% of our compute
vs best tuned Transformer so far (2 tuned runs)	1.996 bpc	1.888 bpc	Better by 0.11 at equal training compute (101%)
vs LSTM-512 / Transformer (90M chars): queued	1.661 / 1.604 bpc	1.857 bpc	Behind, at under 1/16 of their training compute
NeuroBench chaotic forecast (20/30 runs)	13.37% (LSTM)	14.37%	Behind by 1.0 point, with an 8.5× smaller model

Wins vs both saved Transformers; a tuned small LSTM still leads. That gap is the next proof.

Compute = estimated arithmetic operations. Baselines trained by us; tuned versions at our budgets are running. Scoreboard: evidence pack R1.

Event-driven models aimed for parity. Ours beats small Transformers on compute.

Published event-driven and spiking language models, as reported by their authors, beside our completed result.

Model	How it computes	Reported language result
EGRU on SpiNNaker2 (2023)	Clocked recurrent net; units fire events above a threshold	Matches LSTMs
SpikeGPT, 216M (2023)	Clocked binary spikes in a recurrent Transformer variant	“Competitive” with non-spiking models
Loihi 2 LLM, 370M (2025); sparse SSM (2026)	Conventional models made sparse/quantized; the chip supplies asynchrony	No accuracy loss / comparable to dense
Timing-coded LLM, 1.5B (2026)	Spike times carry values; closest to our races	“Clear gap” in language perplexity
Sleeping Machines (ours)	Learned arrival times choose which memories update; unchosen paths learn too	Beats two small Transformers at 40% of training compute; tuned LSTMs lead by 0.06

To our knowledge, the first event-timed model to beat Transformer baselines on language with less compute, so far at small scale.

Different datasets and scales; not a ranking. Ours: text8, 10M characters, small models, single runs, our baselines; LSTM-512 still ahead.

Model capacity and the cost of using it must scale differently.

The opportunity: preserve useful intelligence while reducing the work paid for each prediction.

MODEL CAPACITY

Know more

Larger models and richer memory can improve capability. Accessing that capacity still costs computation and data movement.

DEPLOYMENT ECONOMICS

Do less work

Serving cost, latency and memory determine whether capability can be deployed in constrained environments.

OUR APPROACH

Select + retain

Learn which local computations to activate; retain local state and train the alternatives that were not selected.

We are developing a model family and execution substrate around this separation.

One model family across today's architectural landscape.

Our approach combines strengths of today's main model types.

Recurrent networks

Carry a running memory forward.
We add addressed memory and timing.

Transformers

Every input attends to every other.
We keep that where it pays off.

Mixture of experts

Activate part of a large model.
We add private memory and timing.

SLEEPING MACHINES / STATEFUL TEMPORAL PROGRAMS

One interface for content, timing, memory address and state, from single units to complete systems.
Target: computation through timing, sparse memory updates, small messages, and learning from unchosen paths.

Statistics / retrieval

Counts and lookups of past data.
Combined with learned features.

Adaptive memory

Models that adapt while in use.
With local, causal updates.

Event-driven systems

Event-driven, neuromorphic systems.
We add trainable messages.

These are design relationships, not benchmark claims. Each combination must still be built and measured.

Choose the right computation in each region.

One family can mix selective temporal programs with rich interaction and causal memory.

01 / UNITS

Local programs

Each unit chooses what to remember, what message to send and when. Timing itself performs computation.



02 / LAYERS

Adaptive work

A layer can activate one message or many: dense where needed, lean elsewhere.



03 / COMPOSITIONS

Shared evidence

Learned features, counts and stored history can all inform the same prediction.



04 / SYSTEMS

Same learner

The same learning rule works in batches or on a live stream, including learning after deployment.

Counterfactual credit: learn from selected and unrealized routes

No global clock is required. Combined models, learning-after-deployment gains and chip energy are still milestones.

A substrate for intelligence that learns across mind and body.

The ambition: connect sensing, action, language and reasoning in one trainable model, across their natural timescales.



Hypothesis: physical experience teaches reusable abstractions; reasoning improves action. Online learning keeps both adapting.

Upside: persistent generalist agents, from datacenter software to embodied intelligence on mobile robots.

First proof: held-out transfer in both directions, with retained skills and matched data/work. AGI is an ambition, not a demonstrated capability.

Messages race. Selected memories update. Alternatives learn.

A simple event loop connects representation, timing and sparse computation.

01 / ENCODE

Signal + delay

A learned function produces a message and its arrival time from input and local memory.



02 / SELECT

Temporal race

Arrival times select a receiver. Separate address keys and delivered values support sparse execution.



03 / REMEMBER

Local state

Update the selected memory and propagate a new message into deeper computation.



04 / LEARN

Alternatives

Use errors to improve message content and timing, including paths that were not selected.

Counterfactual credit: learn from selected and unrealized routes

Implemented and trained today in ordinary software (CPU); no special hardware needed to run it.

The language experiments establish three useful facts.

Wikipedia text (text8), 10M characters of training. Prediction error in bits per character: lower is better.

IT LEARNS REAL TEXT

1.888 bits/char

Better than the 4-layer Transformer baseline (1.908) with 40% of its training compute. The larger LSTM (1.799) is still ahead.

TRAINING UNCHOSEN PATHS HELPS

−0.135 bits/char

Giving learning signal to the paths the model did not take cuts the error of the same small model, for 0.3% more training compute.

MORE MEMORY, SAME RUNNING WORK

16 → 32 slots

Doubling memory slots lowers the error while each input still updates only 8 slots. Training cost rises; running cost barely changes.

Single runs on one dataset. The next proof is repeated runs plus a measured cost advantage.

Single pass: error versus training compute.

Each model sees the 10M training characters once. Lower and further left is better. Slide 2 adds multi-pass runs.



BEST SINGLE PASS

2.162 bpc

Edges LSTM-256 (2.171), with 2.9× its training compute.

TRAINING COMPUTE

15.4× less

Small model vs 2-layer Transformer: less compute and lower error.

Baselines trained by us on the same data and test set. Compute is estimated arithmetic (trillion operations), not measured energy or cost. Single runs.

Start with low-batch language inference on existing hardware.

Product hypothesis for local and on-premise text applications; no customer interest reported yet.

WHO WOULD EVALUATE IT

Platform teams

Teams deploying small language models under latency, memory or cost constraints. Demand validation is a next step.

WHAT WE WOULD SUPPLY

Model + runtime

A trained model, sparse inference runtime and a reproducible quality-and-cost evaluation on the intended workload.

WHY THEY WOULD ADOPT

Verified savings

Adoption depends on equivalent useful quality, service reliability and a measured reduction in complete deployment cost.

Commercial gate: target $\geq 20\%$ full-system cost reduction at matched quality on one selected workload.

Software proves the advantage. Hardware multiplies it.

One research program, with staged routes to deployment.

01 / MODELS + RUNTIME

First product

Validate the language workload on existing hardware. License a model/runtime with auditable deployment economics.

02 / MEMORY-LOCAL ACCELERATORS

5.8× less data moved

Modeled per character: our 1.888-bpc model moves 5.8× fewer memory bytes than the Transformer it beats. 10× more memory adds only 5% traffic. Moving data dominates chip energy.

03 / CLOCKLESS SILICON

Work only on events

No global clock: idle parts draw almost no power, answers arrive as soon as they are ready, and cool chips allow dense 3D stacking. Also the missing model for neuromorphic chips.

The €3M round funds scale and a hardware cost model of the trained network; silicon follows measured proof.

A useful alternative could strengthen a hardware ecosystem.

Potential evaluation and licensing routes; no partner interest or relationship is claimed.

AMD

Model + platform

Its ~\$665M Silo AI acquisition and Liquid AI collaboration show the value of model/software capability on AMD hardware.

INTEL

Event execution

Hala Point and Loihi 2 provide a relevant research environment for expressive event models and continuous learning.

GOOGLE

Full-stack design

TPU model-hardware co-design creates a demanding evaluation venue for quality, memory and execution improvements.

Partner proposition: demonstrate a differentiated workload advantage that improves platform adoption.

Price the runtime against verified customer value.

Illustrative annual license economics; assumptions to test with an evaluation partner.

Per-customer assumption	Calculation	Annual amount
Eligible workload cost	Existing deployment cost we can actually influence	€1.00M
Gross savings	20% complete-cost reduction at matched quality	€200k
Runtime license	20% of verified gross savings	€40k

Customer retains €160k/year before migration and other costs. At the same assumed price, 25 deployments would generate €1M annual license revenue.

Validate pricing through matched quality, reliable operation and measured customer benefit.

AI research and software architecture in one founder.

Tero Keski-Valkama · Sole founder · Public career and invention record.

Tero Keski-Valkama

Sole founder

Experience spans applied ML, multimodal foundation models and software architecture. The round adds specialist capacity around the founder.

2012-2018

Cybercom · Software Architect

Historical role in public professional profile.

2018-2022

HERE · Lead ML & AI Engineer

Self-reported research and applied ML leadership.

2022-2025

Alloy.ai → Kaiko.ai

Staff engineering; multimodal foundation-model work.

2023 publication

EP4148389A2 · Named inventor

Maps from geospatial observations; applicant HERE.

Historical roles are self-reported; inventorship is documented. Planned hiring: learning research, runtime engineering, evaluation and measurement.

Build defensibility across the model and its execution.

The strongest potential asset is an integrated stack that is difficult to reproduce and deploy.

MODEL + LEARNING

Research know-how

Timing, state and alternative-credit methods, supported by retained ablations and numerical contracts.

RUNTIME + HARDWARE

Execution know-how

Sparse kernels, memory management, measurement and hardware mapping can reinforce the model advantage.

RIGHTS + INTEGRATION

Company assets

Establish contribution rights and licensing, then develop tooling and partner-specific deployment expertise.

Private development supports controlled disclosure. IP ownership and commercial integrations remain diligence items.

Three gates determine whether this becomes a business.

Each gate has a falsifiable result and a consequence for further spending.

01 / QUALITY AT SCALE

Replicate

At least three runs per result, fixed held-out tests and tuned baselines. Compare quality and total compute; publish failures too.

02 / COMPLETE ECONOMICS

Measure

Show the fast sparse runtime gives the trained model's answers. Measure serving cost, latency, memory and energy on one workload.

03 / ADOPTION

Evaluate

Identify a use case and an evaluator willing to test it. Confirm useful quality, integration effort and willingness to pay.

If a gate fails, diagnose or narrow the workload before committing more compute or new silicon.

The next 18 months produce evidence investors can underwrite.

Proposed milestones measured from funding; spending advances with completed gates.

MONTHS 0-3

Team + proofs

Hire the initial team. Establish company/IP rights. Verify the fast sparse runtime on trained models; repeat results on larger data.

MONTHS 3-6

One workload

Run tuned controls and measure total cost, latency, memory and energy. Select a workload only if quality and economics hold.

MONTHS 6-12

External trial

Seek independent reproduction and prospective customer evaluations. Test pricing, integration effort and willingness to pay.

MONTHS 12-18

Product + scale

Turn a validated workload into a deployable runtime. Expand models and assess hardware mapping for the next financing stage.

Next-round objective: replicated quality, a measured cost advantage and a credible evaluation pathway.

€3M funds an 18-month research and engineering program.

A proposed six-person average team, staged compute and hardware feasibility.



- Team: research + systems **€1.35M | 45.0%**
- Compute + independent replication **€0.90M | 30.0%**
- Hardware feasibility + measurement **€0.25M | 8.3%**
- IP, legal, operations **€0.20M | 6.7%**
- Contingency reserve **€0.30M | 10.0%**

€2.70M operating plan + €0.30M reserve · Planned operating burn averages €150k/month.
Budget assumptions: six average FTE including founder at €150k/year fully loaded; compute and hiring quotes remain to be obtained.

Raise €3M to establish a commercially valuable platform.

Proposed priced round; terms remain subject to company formation and investor diligence.

CAPITAL TO RAISE

€3M

18 months of research, systems engineering and staged validation.

PROPOSED PRE-MONEY

€50M

A premium research-stage proposal for the model/runtime/substrate opportunity.

INITIAL OWNERSHIP

5.66%

€53M post-money, before option-pool changes, preferences and later dilution.

The proposal prices a model/learning family, sparse runtime and future chip architecture that already beats Transformer baselines at a fraction of the compute. A confirmed 90M-character win and a NeuroBench first place would justify a higher next price (the €100M scenario in the appendix).

Valuation sensitivities and conditional exit scenarios are in the appendix.

Build the model. Prove the economics. Open the hardware path.

€3M proposed raise · €50M proposed pre-money · Private investor discussion.

€3 million

Already beating Transformer baselines on compute.
This round tests whether the advantage scales.

- 01** Replicated language quality and learning mechanisms.
- 02** A deployable runtime with measured system economics.
- 03** External evaluation and a credible path to hardware.

Tero Keski-Valkama · Private investor diligence available by arrangement